CS353 Database Systems - Spring 2026

Design Report

24.03.2026

Group 12

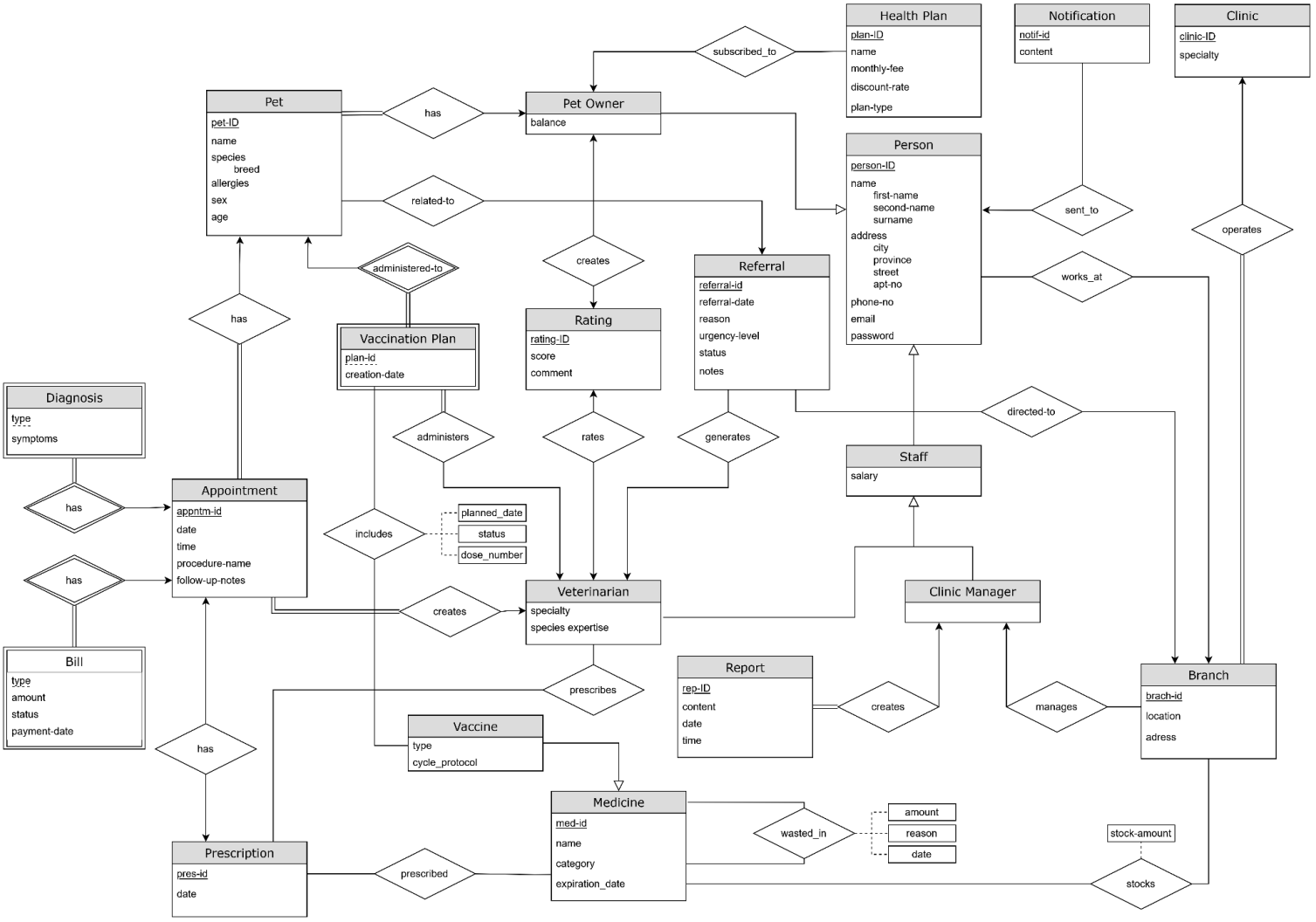
Hasan Tufan - 22203594
Lütfü Heysem Kızıloğlu - 22301926
Nijat Mukhtarov - 22301195
Eser Tekin Tekeli - 22301742
Emir Said Bakan - 22302852

Contents

1. Revised E/R Diagram 1
2. Table Schemas 1
2.1. Person and Staff Related Tables 1
2.2.1 Person 1
2.2.2 Pet-owner 2
2.2.3 Staff 2
2.2.4 Clinic-manager 3
2.2.5 Veterinarian 3
2.2. Clinic and Branch Related Tables 3
2.3.1 Clinic 3
2.3.2 Branch 4
2.3. Pet and Appointment Related Tables 4
2.4.1 Health Plan 4
2.4.2 Pet 5
2.4.3 Appointment 5
2.4.4 Diagnosis 6
2.4. Medicine and Stock Related Tables 6
2.5.1 Medicine 6
2.5.2 Vaccine 6
2.5.3 Branch-Stock 7
2.5.4 Prescription 7
2.5.5 Prescription-Medicine 8
2.5.5 Vaccination-plan 8
2.5.6 Vaccination-Plan-Includes 8
2.5. Additional Functionality Related Tables 9
2.6.1 Bill 9
2.6.2 Referral 9
2.6.3 Rating 10
2.6.4 Waste 10
2.6.5 Report 11
2.6.6 Notification 11
3. User Interface Design and Corresponding SQL Statements 12
3.1. Login and Registration Interfaces 12
3.2. Additional Functionality 15
3.2.1. Clinic Manager Interface: Managing Health Plans 15
3.2.2. Pet Owner Interface: Viewing and Subscribing to Plans 17
3.3. Topic-Specific Functionality: Booking an Appointment & Managing Patient Records 18
3.3.1. Pet Owner Interface: Booking an Appointment 19
3.3.2. Veterinarian Interface: Managing Patient Records 20

3.4. Advanced Database Components 22
3.4.1. Views 22
3.4.2. Triggers 23
3.4.3. Constraints 24
4. Implementation Plan 24
4.1. DBMS 24
4.2. Backend 25
4.3. Frontend 25
4.4. Raw SQL Usage 25
4.5. Planned Modules 25

1. Revised E/R Diagram



2. Table Schemas

2.1. Person and Staff Related Tables

2.2.1 Person

Person(person-ID, first-name, second-name, surname, city, province, street, apt-no, phone-no, email, password)

Domains:

person-ID: INT NOT NULL UNIQUE

first-name: VARCHAR(50) NOT NULL

second-name: VARCHAR(50)

surname: VARCHAR(50) NOT NULL

city: VARCHAR(50)

province: VARCHAR(50)

street: VARCHAR(100)

apt-no: VARCHAR(20)

phone-no: VARCHAR(20) NOT NULL

email: VARCHAR(100) NOT NULL UNIQUE

password: VARCHAR(50) NOT NULL

Foreign Keys: -

Candidate Keys: person-ID, email

Primary Key: person-ID

2.2.2 Pet-owner

Pet-owner(person-ID, balance, plan-ID)

Domains:

person-ID: INT NOT NULL UNIQUE (FK → Person.person-ID)

balance: NUMERIC(10,2) NOT NULL

plan-ID: INT (FK → Health-plan.plan-ID)

Foreign Keys: person-ID → Person(person-ID), plan-ID → Health-plan(plan-ID)

Candidate Keys: person-ID

Primary Key: person-ID

2.2.3 Staff

Staff(person-ID, salary, branch-id)

Domains:

person-ID: INT NOT NULL UNIQUE (FK → Person.person-ID)

salary: NUMERIC(10,2) NOT NULL

branch-id: INT NOT NULL (FK → Branch.branch-id)

Foreign Keys: person-ID → Person(person-ID), branch-id → Branch(branch-id)

Candidate Keys: person-ID

Primary Key: person-ID

2.2.4 Clinic-manager

Clinic-manager(person-ID)

Domains:

person-ID: INT NOT NULL UNIQUE (FK → Staff.person-ID)

Foreign Keys: person-ID → Staff(person-ID)

Candidate Keys: person-ID

Primary Key: person-ID

2.2.5 Veterinarian

Veterinarian(person-ID, specialty, species-expertise)

Domains:

person-ID: INT NOT NULL UNIQUE (FK → Staff.person-ID)

specialty: VARCHAR(100) NOT NULL

species-expertise: VARCHAR(100) NOT NULL

Foreign Keys: person-ID → Staff(person-ID)

Candidate Keys: person-ID

Primary Key: person-ID

2.2. Clinic and Branch Related Tables

2.3.1 Clinic

Clinic(clinic-ID, specialty)

Domains:

clinic-ID: INT NOT NULL UNIQUE

specialty: VARCHAR(100)

Foreign Keys: -

Candidate Keys: clinic-ID

Primary Key: clinic-ID

2.3.2 Branch

Branch(branch-id, location, address, clinic-ID, manager-ID)

Domains:

branch-id: INT NOT NULL UNIQUE

location: VARCHAR(100) NOT NULL

address: VARCHAR(250) NOT NULL

clinic-ID: INT NOT NULL (FK → Clinic.clinic-ID)

manager-ID: INT (FK → Clinic-manager.person-ID)

Foreign Keys: clinic-ID → Clinic(clinic-ID), manager-ID → Clinic-manager(person-ID)

Candidate Keys: branch-id

Primary Key: branch-id

2.3. Pet and Appointment Related Tables

2.4.1 Health Plan

Health-plan(plan-ID, name, monthly-fee, discount-rate, plan-type)

Domains:

plan-ID: INT NOT NULL UNIQUE

name: VARCHAR(50) NOT NULL

monthly-fee: NUMERIC(10,2) NOT NULL

discount-rate: NUMERIC(4,2) NOT NULL

plan-type: VARCHAR(50) NOT NULL

Foreign Keys: -

Candidate Keys: plan-ID

Primary Key: plan-ID

2.4.2 Pet

Pet(pet-ID, name, species, breed, allergies, sex, age, owner-ID)

Domains:

pet-ID: INT NOT NULL UNIQUE

name: VARCHAR(50) NOT NULL

species: VARCHAR(50) NOT NULL

breed: VARCHAR(50)

allergies: VARCHAR(250)

sex: VARCHAR(10) NOT NULL

Age: INT

owner-ID: INT NOT NULL (FK → Pet-owner.person-ID)

Foreign Keys: owner-ID → Pet-owner(person-ID)

Candidate Keys: pet-ID

Primary Key: pet-ID

2.4.3 Appointment

Appointment(appntm-id, date, time, procedure-name, follow-up-notes, pet-ID, vet-ID)

Domains:

appntm-id: INT NOT NULL UNIQUE

date: DATE NOT NULL

time: TIME NOT NULL

procedure-name: VARCHAR(100) NOT NULL

follow-up-notes: VARCHAR(500)

pet-ID: INT NOT NULL (FK → Pet.pet-ID)

vet-ID: INT NOT NULL (FK → Veterinarian.person-ID)

Foreign Keys: pet-ID → Pet(pet-ID), vet-ID → Veterinarian(person-ID)

Candidate Keys: appntm-id

Primary Key: appntm-id

2.4.4 Diagnosis

Diagnosis(appntm-id, type, symptoms)

Domains:

appntm-id: INT NOT NULL (FK → Appointment.appntm-id)

type: VARCHAR(50) NOT NULL

symptoms: VARCHAR(500) NOT NULL

Foreign Keys: appntm-id → Appointment(appntm-id)

Primary Key: appntm-id, type

2.4. Medicine and Stock Related Tables

2.5.1 Medicine

Medicine(med-id, name, category, expiration-date)

Domains:

med-id: INT NOT NULL UNIQUE

name: VARCHAR(100) NOT NULL

category: VARCHAR(100) NOT NULL

expiration-date: DATE NOT NULL

Foreign Keys: -

Candidate Keys: med-id

Primary Key: med-id

2.5.2 Vaccine

Vaccine(med-id, type, cycle-protocol)

Domains:

med-id: INT NOT NULL UNIQUE (FK → Medicine.med-id)

type: VARCHAR(100) NOT NULL

cycle-protocol: VARCHAR(250) NOT NULL

Foreign Keys: med-id → Medicine(med-id)

Candidate Keys: med-id

Primary Key: med-id

2.5.3 Branch-Stock

Branch-stock(branch-id, med-id, stock-amount)

Domains:

branch-id: INT NOT NULL (FK → Branch.branch-id)

med-id: INT NOT NULL (FK → Medicine.med-id)

stock-amount: INT NOT NULL

Foreign Keys: branch-id → Branch(branch-id), med-id → Medicine(med-id)

Candidate Keys: (branch-id, med-id)

Primary Key: (branch-id, med-id)

2.5.4 Prescription

Prescription(pres-id, date, appntm-id, vet-ID)

Domains:

pres-id: INT NOT NULL UNIQUE

date: DATE NOT NULL

appntm-id: INT NOT NULL (FK → Appointment.appntm-id)

vet-ID: INT NOT NULL (FK → Veterinarian.person-ID)

Foreign Keys: appntm-id → Appointment(appntm-id), vet-ID → Veterinarian(person-ID)

Candidate Keys: pres-id

Primary Key: pres-id

2.5.5 Prescription-Medicine

Prescription-medicine(pres-id, med-id)

Domains:

pres-id: INT NOT NULL (FK → Prescription.pres-id)

med-id: INT NOT NULL (FK → Medicine.med-id)

Foreign Keys: pres-id → Prescription(pres-id), med-id → Medicine(med-id)

Candidate Keys: (pres-id, med-id

Primary Key: (pres-id, med-id)

2.5.5 Vaccination-plan

Vaccination-plan(plan-id, creation-date, pet-ID, vet-ID)

Domains:

plan-id: INT NOT NULL UNIQUE

creation-date: DATE NOT NULL

pet-ID: INT NOT NULL (FK → Pet.pet-ID)

vet-ID: INT NOT NULL (FK → Veterinarian.person-ID)

Foreign Keys: pet-ID → Pet(pet-ID), vet-ID → Veterinarian(person-ID)

Candidate Keys: plan-id

Primary Key: plan-id

2.5.6 Vaccination-Plan-Includes

Vaccination-Plan-Includes(plan-id, med-id, planned_date, status, dose_number)

Domains:

plan-id: INT NOT NULL (FK → Vaccination-plan.plan-id)

med-id: INT NOT NULL (FK → Vaccine.med-id)

planned_date: DATE NOT NULL

status: VARCHAR(50) NOT NULL

dose_number: INT NOT NULL

Foreign Keys: plan-id → Vaccination-plan(plan-id), med-id → Vaccine(med-id)

Primary Key: (plan-id, med-id)

2.5. Additional Functionality Related Tables

2.6.1 Bill

Bill(bill-id, appntm-id, type, amount, status, payment-date)

Domains:

bill-id: INT NOT NULL UNIQUE

appntm-id: INT NOT NULL (FK → Appointment.appntm-id)

type: VARCHAR(50) NOT NULL

amount: NUMERIC(10,2) NOT NULL

status: VARCHAR(50) NOT NULL

payment-date: DATE

Foreign Keys: appntm-id → Appointment(appntm-id)

Candidate Keys: bill-id

Primary Key: (appntm-id, bill-id)

2.6.2 Referral

Referral(referral-id, referral-date, reason, urgency-level, status, notes, vet-ID, branch-id, pet-ID)

Domains:

referral-id: INT NOT NULL UNIQUE

referral-date: DATE NOT NULL

reason: VARCHAR(250) NOT NULL

urgency-level: VARCHAR(50) NOT NULL

status: VARCHAR(50) NOT NULL

notes: VARCHAR(500)

vet-ID: INT NOT NULL (FK → Veterinarian.person-ID)

branch-id: INT NOT NULL (FK → Branch.branch-id)

pet-ID: INT NOT NULL (FK → Pet.pet-ID)

Foreign Keys: vet-ID → Veterinarian(person-ID), branch-id → Branch(branch-id), pet-ID → Pet(pet-ID)

Candidate Keys: referral-id

Primary Key: referral-id

2.6.3 Rating

Rating(rating-ID, score, comment, owner-ID, vet-ID)

Domains:

rating-ID: INT NOT NULL UNIQUE

score: INT NOT NULL CHECK (score >= 1 AND score <= 5)

comment: VARCHAR(500)

owner-ID: INT NOT NULL (FK → Pet-owner.person-ID)

vet-ID: INT NOT NULL (FK → Veterinarian.person-ID)

Foreign Keys: owner-ID → Pet-owner(person-ID), vet-ID → Veterinarian(person-ID)

Candidate Keys: rating-ID, (owner-ID, vet-ID)

Primary Key: rating-ID

2.6.4 Waste

Waste(waste-id, amount, reason, date, med-id, branch-id)

Domains:

waste-id: INT NOT NULL UNIQUE

amount: NUMERIC(10,2) NOT NULL

reason: VARCHAR(250) NOT NULL

date: DATE NOT NULL

med-id: INT NOT NULL (FK → Medicine.med-id)

branch-id: INT NOT NULL (FK → Branch.branch-id)

Foreign Keys: med-id → Medicine(med-id), branch-id → Branch(branch-id)

Candidate Keys: waste-id

Primary Key: waste-id

2.6.5 Report

Report(rep-ID, content, date, time, manager-ID)

Domains:

rep-ID: INT NOT NULL UNIQUE

content: VARCHAR(1000) NOT NULL

date: DATE NOT NULL

time: TIME NOT NULL

manager-ID: INT NOT NULL (FK → Clinic-manager.person-ID)

Foreign Keys: manager-ID → Clinic-manager(person-ID)

Candidate Keys: rep-ID Primary Key: rep-ID

2.6.6 Notification

Notification(notif-id, content,, person-ID)

Domains:

notif-id: INT NOT NULL UNIQUE

content: VARCHAR(250) NOT NULL

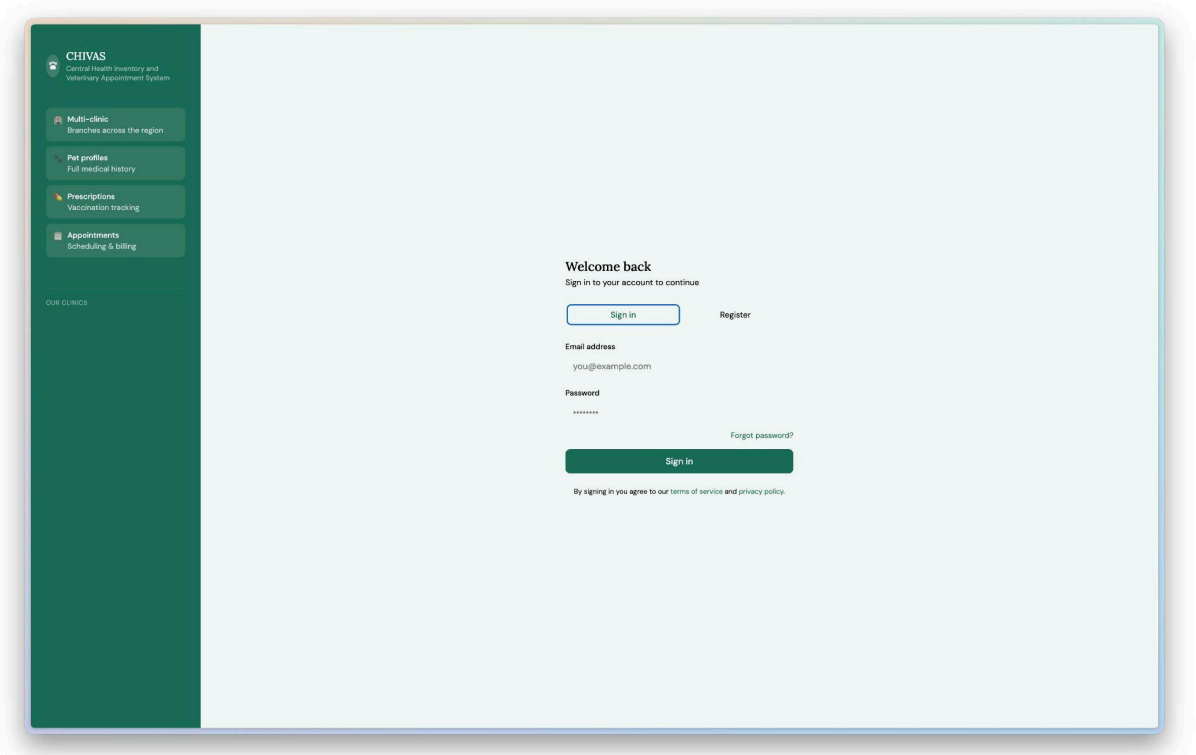
person-ID: INT NOT NULL (FK → Person.person-ID)

Foreign Keys: person-ID → Person(person-ID)

Candidate Keys: notif-id Primary Key: notif-id

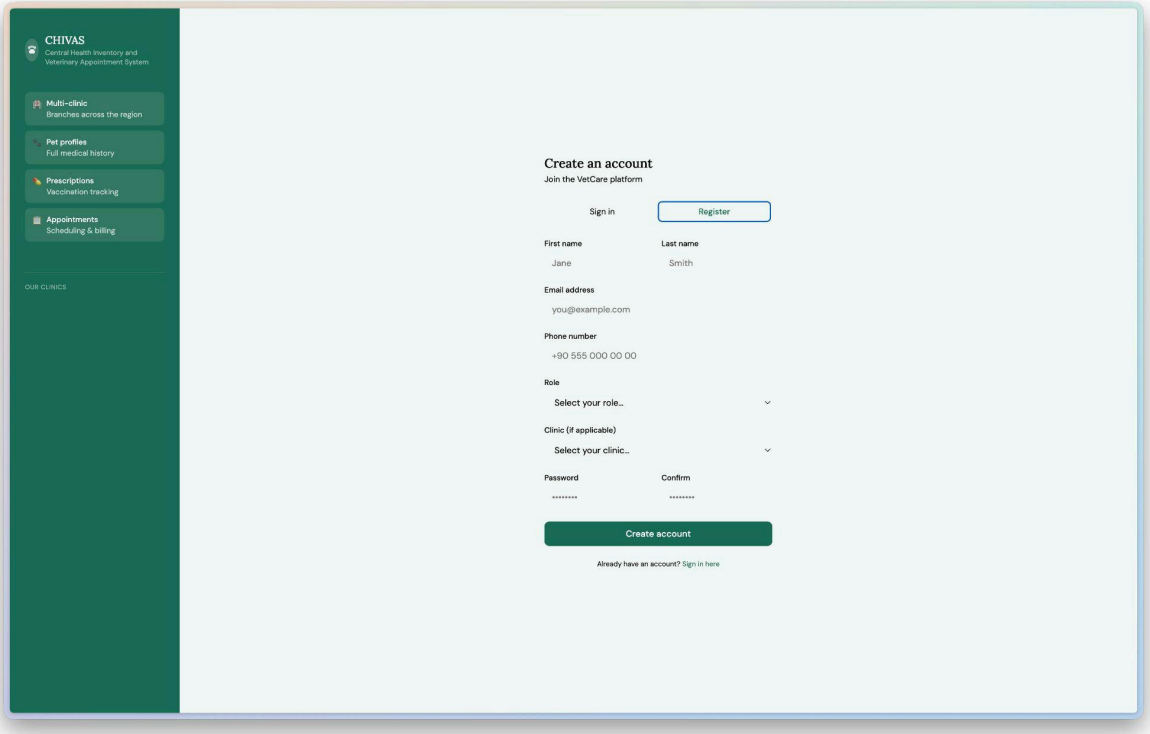
3. User Interface Design and Corresponding SQL Statements

3.1. Login and Registration Interfaces



```
SELECT
  person_ID,
  first_name,
  second_name,
  surname,
  email,
  phone_no,
  city,
  province,
  street,
  apt_no
FROM Person
WHERE LOWER(email) = LOWER('[email]');

SELECT
  c.clinic_ID,
  c.specialty,
  b.branch_ID,
  b.location,
  b.adress
FROM Clinic c
JOIN Branch b ON b.clinic_ID = c.clinic_ID
ORDER BY c.clinic_ID, b.location;
```

```
SELECT COUNT(*) AS email_count
FROM Person
WHERE LOWER(email) = LOWER('[email]');
```


CHIVAS
Central Health Inventory and Veterinary Appointment System
Multi-clinic
Branches across the region
Pet profiles
Full medical history
Prescriptions
Vaccination tracking
Appointments
Scheduling & billing
OUR CLINICS
Create an account
Join the VetCare platform
Sign in
Register
First name
Jane
Last name
Smith
Email address
you@example.com
Phone number
+90 555 000 00 00
Role
Select your role...
Clinic (if applicable)
Select your clinic...
Password
Confirm
Create account
Already have an account? Sign in here

```
INSERT INTO Person (
   person_ID,
   first_name,
   second_name,
   surname,
   email,
   phone_no,
   city,
   province,
   street,
   apt_no,
   password
) VALUES (
   '[person_ID]',
   '[first_name]',
   '[second_name]',
   '[surname]',
   LOWER('[email]'),
   '[phone_no]',
   '[city]',
   '[province]',
   '[street]',
   '[apt_no]',
   '[hashed_password]'
);
INSERT INTO Pet_Owner (
   person_ID,
   balance
) VALUES (
   '[person_ID]',
   0.00
);


INSERT INTO Staff (
   person_ID,
   salary
) VALUES (
   '[person_ID]',
   '[salary]'
);

INSERT INTO Veterinarian (
   person_ID,
   specialty,
   species_expertise
```

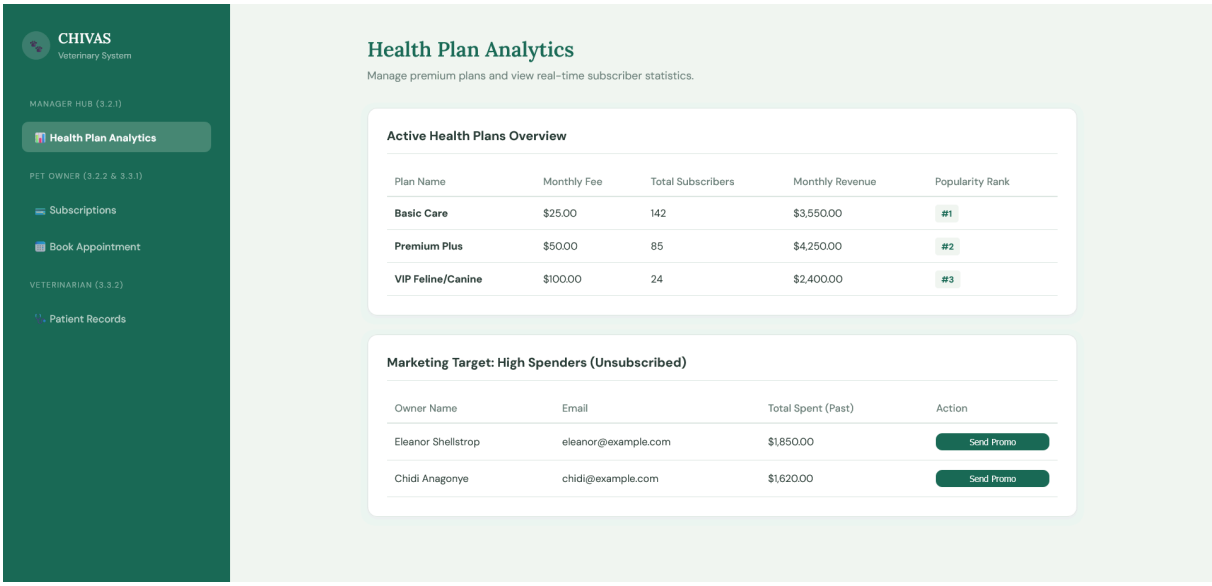
```
) VALUES (
   '[person_ID]',
   '[specialty]',
   '[species_expertise]'
);


INSERT INTO Clinic_Manager (
   person_ID
) VALUES (
   '[person_ID]'
);
INSERT INTO works_at (
   person_ID,
   branch_ID
) VALUES (
   '[person_ID]',
   '[branch_ID]'
);
```

3.2. Additional Functionality

This section introduces the Premium Health Plans module, an additional functionality designed to increase interaction between the clinic and pet owners while providing financial advantages. Clinic managers can create and manage these plans, and pet owners can subscribe to them to receive discounts on treatments.

3.2.1. Clinic Manager Interface: Managing Health Plans

This interface can only be accessed by Clinic Managers. There is a dashboard that shows a data grid/table of all existing health plans in the system. Under this data grid/table, there is a "Create New Health Plan" form that has fields for Plan Name, Monthly Fee, Discount Rate, and Plan Type. There are also "Save" and "Update" buttons.



Corresponding SQL Statements:

-**Fetch health plans with advanced statistics (Using `LEFT JOIN`, `GROUP BY`, and Window Functions):** *This query calculates the total subscribers and revenue per plan, and ranks them based on popularity.*

```
CREATE VIEW Active_Health_Plan_Stats AS
SELECT
    hp.plan_ID,
    hp.name,
    hp.monthly_fee,
    COUNT(po.person_ID) AS total_subscribers,
    COALESCE(SUM(hp.monthly_fee), 0) AS monthly_revenue,
    RANK() OVER(ORDER BY COUNT(po.person_ID) DESC) AS popularity_rank
FROM Health_plan hp
LEFT JOIN Pet_owner po ON hp.plan_ID = po.plan_ID
GROUP BY hp.plan_ID, hp.name, hp.monthly_fee;
```

Instead of writing complex JOINs, the backend application now simply executes the following query to retrieve the dashboard data:

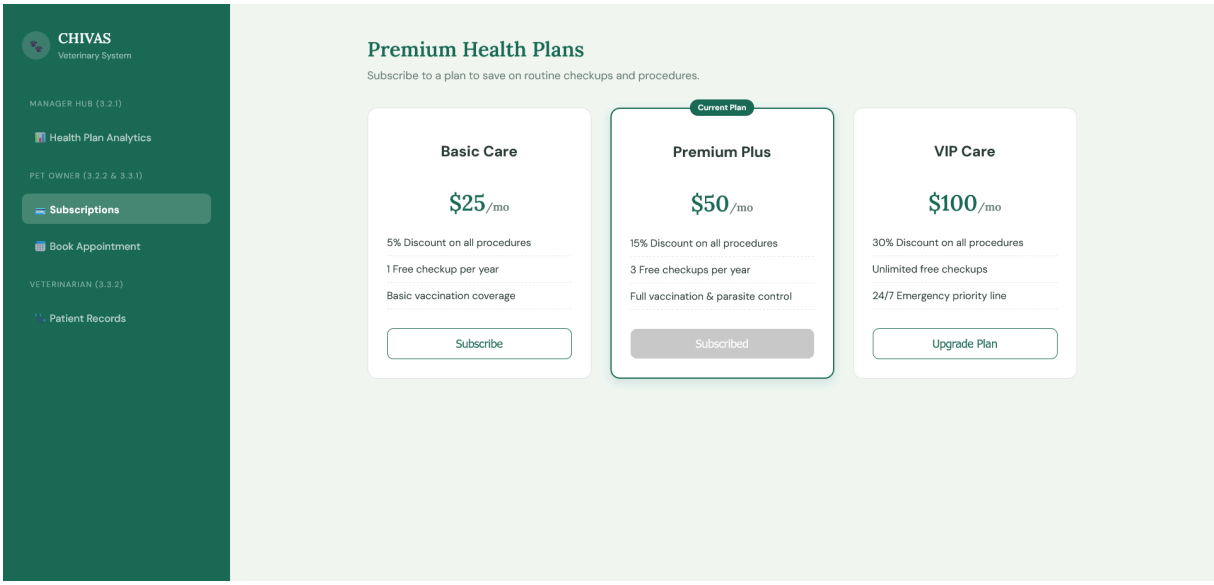
```
SELECT * FROM Active_Health_Plan_Stats ORDER BY popularity_rank ASC;
```

-**Identify Unsubscribed High-Spenders for Marketing (Using Multiple `JOIN`s and `HAVING`):** This query finds pet owners without a health plan who have spent more than $1500 in total on past appointments.

```
SELECT
   p.first_name,
   p.surname,
   p.email,
   SUM(b.amount) AS total_spent
FROM Person p
JOIN Pet_owner po ON p.person_ID = po.person_ID
JOIN Pet pt ON po.person_ID = pt.owner_ID
JOIN Appointment a ON pt.pet_ID = a.pet_ID
JOIN Bill b ON a.appntm_id = b.appntm_id
WHERE po.plan_ID IS NULL AND b.status = 'Paid'
GROUP BY p.person_ID, p.first_name, p.surname, p.email
HAVING SUM(b.amount) > 1500.00
ORDER BY total_spent DESC;
```

3.2.2. Pet Owner Interface: Viewing and Subscribing to Plans

The interface displays the available health plans as "Pricing Cards" to the logged-in Pet Owner. Instead of simply displaying the available plans, the system also checks whether the user is already subscribed to one of the plans, in which case it would be displayed as "Current Plan." Once the user has chosen to subscribe to a plan, the system would then proceed to conduct a secure transaction to update the user's account and deduct the initial monthly fee from their clinic balance.



Corresponding SQL Statements:

-**Dynamic Plan Display with Current Status (Using CASE and Correlated Subquery):** *This query lists all plans and dynamically evaluates if the logged-in user is already a member of each specific plan.*

```
SELECT
   hp.plan_ID,
   hp.name,
   hp.monthly_fee,
   hp.discount_rate,
   CASE
      WHEN hp.plan_ID = (SELECT plan_ID FROM Pet_owner WHERE person_ID
= '[logged_in_person_ID]') THEN 'Current Plan'
      ELSE 'Available'
   END AS subscription_status
FROM Health_plan hp
ORDER BY hp.monthly_fee ASC;
```

-***Secure Plan Subscription Process (Using `TRANSACTION`):*** *To ensure data integrity (ACID properties), updating the user's plan and deducting the initial fee from their balance is wrapped in a single transaction block.*

```
BEGIN TRANSACTION;

-- 1. Update the owner's assigned health plan
UPDATE Pet_owner
SET plan_ID = '[selected_plan_ID]'
WHERE person_ID = '[logged_in_person_ID]';

-- 2. Deduct the monthly fee from the owner's balance
UPDATE Pet_owner
SET balance = balance - (SELECT monthly_fee FROM Health_plan WHERE
plan_ID = '[selected_plan_ID]')
WHERE person_ID = '[logged_in_person_ID]';

COMMIT;
```

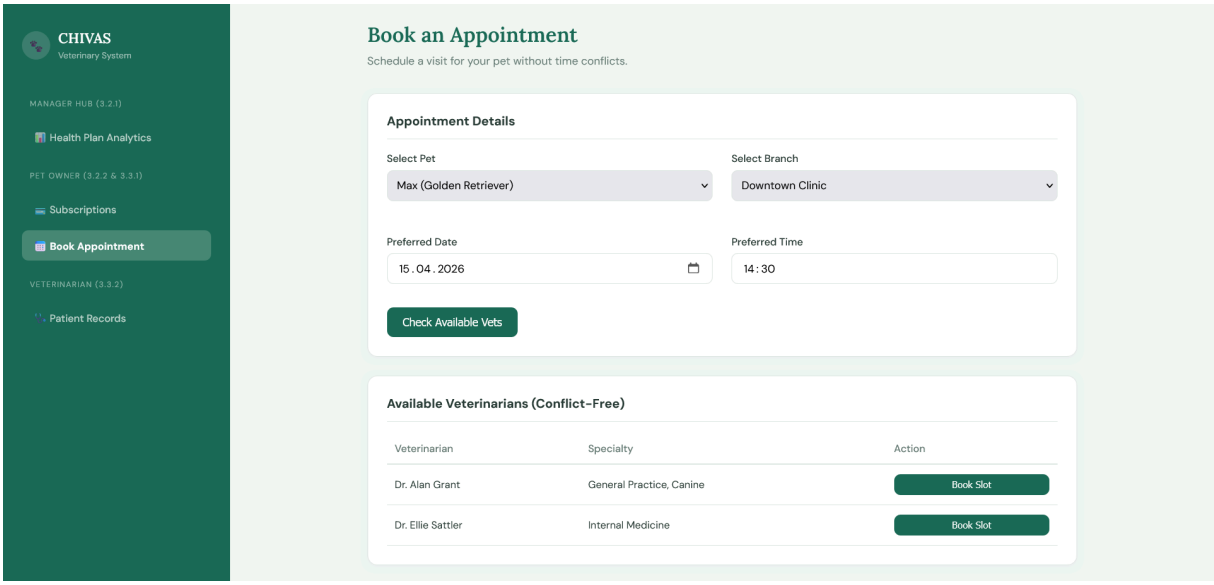
3.3. Topic-Specific Functionality: Booking an Appointment & Managing Patient Records

In this section, we discuss the basic operations of a veterinary clinic, such as booking appointments without time conflicts and providing veterinarians with detailed medical histories of their patients. To implement this, we use advanced SQL features such as NOT EXISTS for avoiding conflicts, STRING_AGG/GROUP_CONCAT for aggregating data, and transactions for ensuring data integrity.

3.3.1. Pet Owner Interface: Booking an Appointment

The pet owner is directed to a "Book Appointment" wizard. The pet owner selects his registered pet, selects a date and a branch, and the system dynamically displays a list of veterinarians available on that specific day.

Once a time slot and a veterinarian are selected, the system confirms the booking and automatically generates a pending bill.



Corresponding SQL Statements:

-**Find Available Veterinarians (Preventing Time Conflicts using `NOT EXISTS`):** *This complex query ensures that a veterinarian is only shown as available if they do NOT already have an appointment scheduled at the user's requested date and time within the selected branch.*

```
SELECT
    v.person_ID AS vet_id,
    p.first_name,
    p.surname,
    v.specialty
FROM Veterinarian v
JOIN Staff s ON v.person_ID = s.person_ID
JOIN Person p ON v.person_ID = p.person_ID
WHERE s.branch_id = '[selected_branch_id]'
  AND NOT EXISTS (
      SELECT 1
```

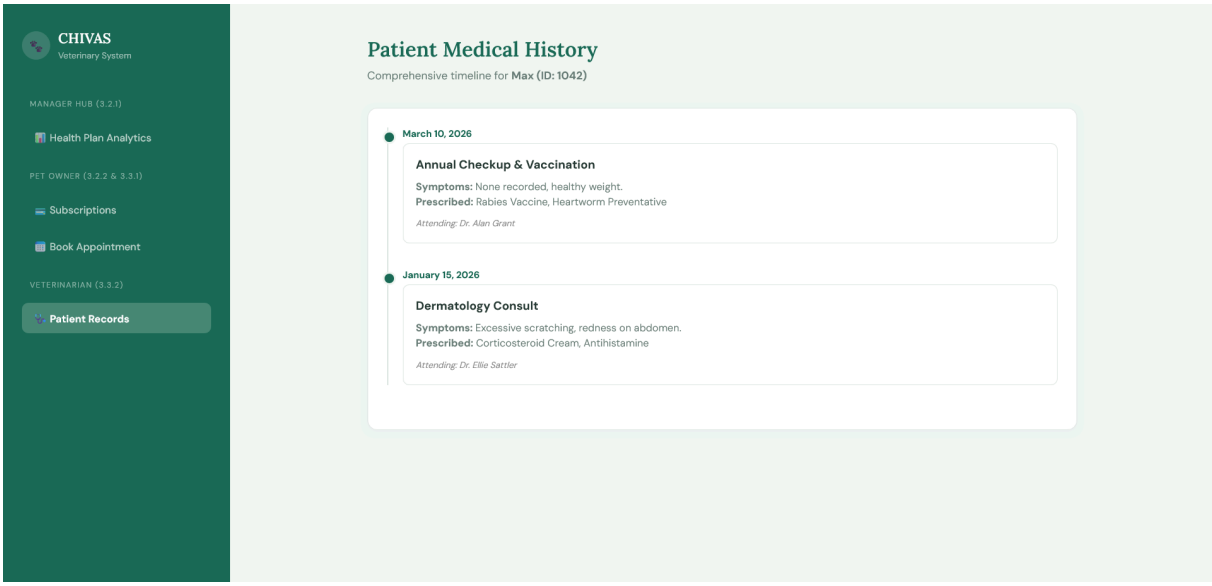
```
    FROM Appointment a
    WHERE a.vet_ID = v.person_ID
      AND a.date = '[requested_date]'
      AND a.time = '[requested_time]'
  );
```

-***Secure Appointment Booking and Billing (Using `TRANSACTION`):*** *Creating an appointment must trigger the creation of a corresponding bill. Wrapping this in a transaction ensures that if the billing fails, the appointment is not created, maintaining ACID properties.*

```
BEGIN TRANSACTION;

-- 1. Create the Appointment Record
INSERT INTO Appointment (
   appntm_id, date, time, procedure_name, pet_ID, vet_ID
) VALUES (
   '[generated_appntm_id]', '[requested_date]', '[requested_time]',
   '[selected_procedure]', '[selected_pet_ID]', '[selected_vet_ID]'
);

-- 2. Generate a Pending Bill for the Procedure
INSERT INTO Bill (
   bill_id, appntm_id, time, amount, status
) VALUES (
   '[generated_bill_id]', '[generated_appntm_id]', '[requested_time]',
   [calculated_procedure_cost], 'Pending'
);

COMMIT;
```

3.3.2. Veterinarian Interface: Managing Patient Records

Upon clicking a scheduled appointment by a veterinarian, they are directed to a page called "Patient Medical Dashboard." This is a single timeline of the entire medical history of a pet, including previous surgeries, symptoms, and all medications administered to the patient, categorized by appointment dates.



Corresponding SQL Statements:

-**Comprehensive Medical History Retrieval (Using Multiple `LEFT JOIN`s and String Aggregation):** *This query joins six different tables to construct a complete medical timeline. It uses an aggregation function (`STRING_AGG` for PostgreSQL or `GROUP_CONCAT` for MySQL) to compress multiple prescribed medicines into a single, readable string per appointment, preventing duplicate rows.*

```
CREATE VIEW Patient_Medical_History AS
SELECT
    a.pet_ID,
    a.date AS visit_date,
    a.procedure_name,
    COALESCE(d.symptoms, 'No symptoms recorded') AS symptoms,
    COALESCE(STRING_AGG(m.name, ', '), 'No Medication Prescribed') AS
prescribed_medicines,
    vp.first_name || ' ' || vp.surname AS attending_vet
FROM Appointment a
LEFT JOIN Diagnosis d ON a.appntm_id = d.appntm_id
LEFT JOIN Prescription pr ON a.appntm_id = pr.appntm_id
LEFT JOIN Prescription_medicine pm ON pr.pres_id = pm.pres_id
LEFT JOIN Medicine m ON pm.med_id = m.med_id
JOIN Person vp ON a.vet_ID = vp.person_ID
WHERE a.date <= CURRENT_DATE
GROUP BY
    a.pet_ID,
    a.appntm_id,
```

```
    a.date,
    a.procedure_name,
    d.symptoms,
    attending_vet;
```

When a veterinarian clicks on a specific patient's profile, the backend quickly retrieves their entire timeline using this simple query:

```
SELECT * FROM Patient_Medical_History
WHERE pet_ID = '[selected_pet_ID]'
ORDER BY visit_date DESC;
```

3.4. Advanced Database Components

In this section, we detail the advanced database features implemented to optimize performance, ensure data integrity, and enforce business rules within the clinic's database system.

3.4.1. Views

- Active_Health_Plan_Stats View

Where we use it: This view is being used in the Clinic Manager Interface in the Health Plan Analytics page.

Why we use it: In order to find the total subscribers and monthly revenue per plan, we have to use complex operations in the Health_plan and Pet_owner tables. We encapsulated this complexity in the database by creating a view. In the backend code, we do not have to use complex operations involving JOINs and GROUP BYs. We simply use this view in our code to get the results, e.g., SELECT * FROM Active_Health_Plan_Stats ORDER BY popularity_rank ASC;.

```
CREATE VIEW Active_Health_Plan_Stats AS
SELECT
    hp.plan_ID,
    hp.name,
    hp.monthly_fee,
    COUNT(po.person_ID) AS total_subscribers,
    COALESCE(SUM(hp.monthly_fee), 0) AS monthly_revenue,
    RANK() OVER(ORDER BY COUNT(po.person_ID) DESC) AS popularity_rank
FROM Health_plan hp
LEFT JOIN Pet_owner po ON hp.plan_ID = po.plan_ID
GROUP BY
    hp.plan_ID,
    hp.name,
```

```
    hp.monthly_fee;
```

- Patient_Medical_History View

Where we use it: We use this view in the Veterinarian Interface when we are looking at the comprehensive medical timeline for a particular patient.

Why we use it: Creating the entire medical timeline entails referencing six separate tables (Appointment, Diagnosis, Prescription, Prescription_medicine, Medicine, and Person) and condensing many types of medicine into one string. By creating a view out of such a structurally complicated query, we are able to simplify the data retrieval process from the frontend. We can easily retrieve a clean timeline with SELECT * FROM Patient_Medical_History WHERE pet_ID = X;.

```
CREATE VIEW Patient_Medical_History AS
SELECT
    a.pet_ID,
    a.date AS visit_date,
    a.procedure_name,
    COALESCE(d.symptoms, 'No symptoms recorded') AS symptoms,
    COALESCE(GROUP_CONCAT(m.name SEPARATOR ', '), 'No Medication
Prescribed') AS prescribed_medicines,
    CONCAT(vp.first_name, ' ', vp.surname) AS attending_vet
FROM Appointment a
LEFT JOIN Diagnosis d ON a.appntm_id = d.appntm_id
LEFT JOIN Prescription pr ON a.appntm_id = pr.appntm_id
LEFT JOIN Prescription_medicine pm ON pr.pres_id = pm.pres_id
LEFT JOIN Medicine m ON pm.med_id = m.med_id
JOIN Person vp ON a.vet_ID = vp.person_ID
WHERE a.date <= CURRENT_DATE
GROUP BY
    a.pet_ID,
    a.appntm_id,
    a.date,
    a.procedure_name,
    d.symptoms,
    vp.first_name,
    vp.surname;
```

3.4.2. Triggers

Automatic Stock Deduction on Waste Logging

Where we use it: This trigger fires when a new record is inserted in the Waste table.

Why we use it: If an employee at a clinic records an amount of medicine wasted, it might be easy for them to forget to manually reduce the amount of that medicine in the system. This trigger maintains data integrity by immediately subtracting the exact amount wasted from the Branch_stock table when a record in the Waste table is inserted.

```
CREATE TRIGGER trigger_deduct_wasted_stock
AFTER INSERT ON Waste
FOR EACH ROW
BEGIN
   UPDATE Branch_stock
   SET stock_amount = stock_amount - NEW.amount
   WHERE branch_id = NEW.branch_id AND med_id = NEW.med_id;
END;
```

3.4.3. Constraints

- Meaningful Rating Score Validation

  Where we use it: On the score column in the Rating table.

Why we use it: In our schema, the score attribute in the database simply has the data type of INT. However, in the absence of this constraint, a bug in the frontend or a malicious user could easily provide a score of -50 or 9000. We use a CHECK constraint to mathematically ensure that our reviews are strictly within the 1-5 star system.

```
ALTER TABLE Rating
ADD CONSTRAINT chk_valid_rating_score
CHECK (score >= 1 AND score <= 5);
```

4. Implementation Plan

The project will be developed using a layered architecture to ensure a clean separation between the user interface, business logic, and data storage.

4.1. DBMS

MySQL will serve as the primary relational database for storing all system data. The database will be structured according to the defined relational schemas, including tables for Person, Pet-owner, Staff, Veterinarian, Clinic, and Appointment. Relational integrity will be enforced using Primary Keys and Foreign Keys. Specific data types and constraints such as NOT NULL for required fields and UNIQUE for identifiers will be implemented at the SQL level.

4.2. Backend

.NET will be used to build the server-side logic and the API layer. The backend will expose REST endpoints to allow the React frontend to perform CRUD operations on the MySQL database. The backend will handle core functionalities like appointment scheduling logic, medical record management, and health plan subscriptions.

4.3. Frontend

React will be used to create a dynamic, single-page application (SPA) for the user interface. The UI will be broken down into reusable components for different user roles, such as the Pet Owner Interface for booking appointments and the Veterinarian Interface for managing patient records. React's state management will be used to handle user sessions and local data display. The frontend will communicate with the .NET backend using HTTP clients to fetch and submit data.

4.4. Raw SQL Usage

For complex operations, such as generating "Comprehensive Medical History" reports involving multiple LEFT JOINs and string aggregations, raw SQL queries will be executed directly via the .NET data access layer to maximize performance.

4.5. Planned Modules

User Management: Implementing registration and login for various roles (Person, Staff, Vet).

Clinic & Branch Management: Managing clinic details and branch-specific stock levels.

Appointment & Medical Records: A module for owners to book appointments and for veterinarians to record diagnoses and prescriptions.

Health & Vaccination Plans: Tools for clinic managers to create health plans and for pet owners to subscribe to them.